

Rushup 営業プラットフォーム

簡易システム仕様書

Version 3.2（更新版）

プロジェクト名	Rushup 営業活動一元管理プラットフォーム
対象企業	株式会社 Rush up
利用人数	30～50 名
作成日	2026 年 5 月 14 日
最終更新日	2026 年 5 月 28 日
ステータス	設計フェーズ（Phase 1 開発準備中）

1. プロジェクト概要

1.1 背景と目的

営業部において、週次報告・KPI 管理・教育進捗などが個別の Excel やツールで管理されており、情報が分散している。本プロジェクトでは、営業活動に関するあらゆるデータを一元管理し、AI 分析による属人化の解消と、上司の教育時間確保を実現する Web アプリケーションを構築する。

1.2 解決すべき課題

- 営業データが分散しており、全体像の把握に時間がかかる
- 上司が部下の課題分析に多くの時間を費やしている
- 営業ノウハウが属人化しており、組織としての改善が困難
- 個人の改善点が数値として可視化されていない
- 稼働データの蓄積・活用ができていない
- KPI が Excel で管理されており、リアルタイムな分析ができない

1.3 ゴールイメージ

- 営業メンバーがアプリを開けば、自分の数字・課題・改善点が一目でわかる
- 上司がダッシュボードで部下全員の状況を瞬時に把握できる
- AI が蓄積データをもとに自動でフィードバック・改善提案を生成する
- 会議資料（PowerPoint）がワンクリックで出力される
- 入社時からの全データが蓄積され、成長履歴として活用できる
- KPI 基準値を個人レベルで設定し、アラートで即座に異常を検知できる
- 時間帯別のパフォーマンス分析で、最も効率の良い行動パターンを特定できる
- 転換率を入力するだけで必要なコール件数が自動計算され、目標設定の工数を削減できる
- 会社・個人の VISION・MISSION をダッシュボードで常時確認できる
- 月次の最終着地（粗利・件数）を翌々月確定後に記録し、正確な成長率を把握できる

2. システム構成

2.1 技術スタック

カテゴリ	技術	用途
フロントエンド	React + Vite	SPA 構築。高速な開発体験とビルド
UI ライブラリ	Lucide React	アイコンコンポーネント
ルーティング	React Router DOM	ネストルーティング・認証ガード
ホスティング	Firebase Hosting	静的ファイル配信（SSL 対応）
データベース	Cloud Firestore	リアルタイム NoSQL DB
認証	Firebase Authentication	メール/Google 認証・ロール管理
サーバーサイド	Cloud Functions	API 処理・スケジュール実行
AI 分析	Claude API / Gemini API	KPI 分析・行動分析・改善提案
帳票出力	pptxgenjs / papaparse	PowerPoint・CSV 出力
外部連携	Microsoft Graph API	Teams 連携・稼働データ取込
バージョン管理	GitHub	コード管理・変更履歴

2.2 ユーザーロール（3 種）

システムのロールは 3 種類とし、画面で見えるデータの範囲で分類する。Firestore の users コレクションには role（3 種類）と title（実際の役職名）の 2 つを持たせる。

ロール	対象役職	閲覧範囲	主な権限
executive	代表・取締役・役員・統括	全社データ	全データ閲覧、設定変更、ユーザー管理、承認
manager	副統括・マネージャー・P マネージャー・サブマネージャー	自組織データ	部下データ閲覧・週次報告・教育記録・承認
leader	TL・一般	個人データのみ	自分のデータ入力・閲覧・AI 分析（自分のみ）

2.3 役職一覧

No.	役職名	ロール	閲覧範囲	備考
1	代表	executive	全体	
2	取締役	executive	全体	
3	役員	executive	全体	
4	統括	executive	全体	
5	副統括	manager	組織数値	
6	マネージャー	manager	組織数値	
7	P マネージャー	manager	組織数値	
8	サブマネージャー	manager	組織数値	
9	TL	leader	個人数値	
10	一般	leader	個人数値	

3. 組織階層設計

3.1 階層構造

組織は複数階層で構成され、各階層の上司は配下の全メンバーのデータを閲覧できる。

- 統括 (executive) → 配下の全副統括・全マネージャー・全メンバーのデータ
 - 副統括 (manager) → 配下の全マネージャー・全メンバーのデータ
 - マネージャー (manager) → 配下の TL・一般メンバーのデータ
 - TL / 一般 (leader) → 自分のデータのみ

3.2 月次の上司・チーム変更への対応

月単位で上司やチーム構成が変わる可能性があるため、Firestore の teams コレクションに期間情報 (startDate, endDate) を持たせる。これにより以下が実現される。

- 現在のチーム構成 → endDate が null のデータを参照
- 過去のデータ閲覧 → 対象月に有効だったチーム構成で紐づけ
- 異動履歴 → 誰がいつどのチームにいたかの履歴が自動保存
- 過去の上司のもとでの実績も正しく紐づいて表示される

4. KPI データ構造

4.1 日次 KPI 入力項目

営業メンバーが毎日入力する項目。現在の Excel 管理と同じ項目をアプリの入力フォームに移行する。

略称	正式名	説明
総	総コール数	その日の総架電数
実	実コール	実際につながったコール数
再	再コール	再架電数
オ	オーナー接触	オーナーに接触できた数
見	見込	見込み客数
ア	アポ	アポイント獲得数
採用	採用数	採用に至った数
訪問件	訪問件数	訪問した件数
受注	受注数	受注した件数
総コ	カウント時間	稼働時間 (コール時間)

4.2 時間帯別 KPI データ

1 時間単位で 6 項目 (総・実・再・オ・見・ア) を 11 時間帯 (8:00~19:00) で記録する。1 日あたり最大 66 データポイント。

時間帯	記録項目	用途
8:00~9:00	総・実・再・オ・見・ア	早朝の活動効率分析

9:00～10:00	総・実・再・オ・見・ア	午前の立ち上がり
10:00～11:00	総・実・再・オ・見・ア	午前のピーク
11:00～12:00	総・実・再・オ・見・ア	午前後半
12:00～13:00	総・実・再・オ・見・ア	昼休み前後
13:00～14:00	総・実・再・オ・見・ア	午後の立ち上がり
14:00～15:00	総・実・再・オ・見・ア	午後のピーク
15:00～16:00	総・実・再・オ・見・ア	午後中盤
16:00～17:00	総・実・再・オ・見・ア	夕方
17:00～18:00	総・実・再・オ・見・ア	終業前
18:00～19:00	総・実・再・オ・見・ア	残業時間帯

4.3 KGI（月次目標設定）

月初に個人ごとに KGI と転換率の基準値を設定する。基準値は本人が提案し、上司（manager）が承認するフローとする。

項目	内容	例
粗利目標	月次の粗利目標金額	400 万円
単価	1 件あたりの平均単価	50 万円
稼働日	その月の営業稼働日数	20 日
受注件数目標	目標受注件数（粗利÷単価）	8 件
完了率	目標完了率	100%
総コール目標	月次の総コール目標数	3,000 コール
転換率（実コール率）	総コールに対する実コール比率	50%
転換率（再コール率）	総コールに対する再コール比率	50%
転換率（オーナー接触率）	実コールに対するオーナー接触比率	25%
転換率（見込率）	オーナー接触に対する見込比率	10%
転換率（アポ率）	オーナー接触に対するアポ比率	目標値（月により変動）
転換率（採用率）	アポに対する採用比率	40～50%
転換率（訪問率）	アポに対する訪問比率	40～50%
転換率（受注率）	訪問に対する受注比率	20～25%

4.4 KGI 承認フロー

- 1. leader（TL・一般）が月初に KGI と転換率の基準値を入力・提案
- 2. manager（上司）に承認リクエストが通知される
- 3. manager が内容を確認し、承認 or 差戻し（コメント付き）
- 4. 差戻しの場合、leader が修正して再提案
- 5. 承認されたらその月の KPI 基準値として確定
- 6. 確定した基準値がアラートの閾値として使用される

5. アラート設計（方針）

KGI と転換率の基準値を個人レベルで設定し、5 パターンのアラートで異常を検知する。アラートは本人と上司の両方に通知される。

アラート種別	検知タイミング	検知内容
日次アラート	毎日リアルタイム / 終業時	日次 KPI 基準値を下回った場合（総コール数、実コール数等）
週次アラート	毎週月曜日	週次の傾向分析（前週比、基準値との乖離）
転換率アラート	日次 / 週次	転換率が基準値から大きく乖離した場合（量ではなく質の問題）
月末着地予測	毎日更新	現在のペースでの月末着地予測が KGI 目標に対して不足する場合
時間帯別分析	日次 / 週次	特定の時間帯でパフォーマンスが著しく低い場合

※ アラートの詳細設計（閾値・通知方法・表示 UI）は別途策定予定

6. データ設計（Firestore コレクション）

コレクション	主なフィールド	書込権限	説明
users	uid, name, email, role, title, teamId, joinDate	executive	ユーザー情報・ロール・役職名・所属チーム
teams	teamId, name, leaderId, members[], parentTeamId, childTeamIds[], startDate, endDate	executive	チーム構成。期間管理で月次変更に対応。parentTeamId で階層化
weeklyReports	userId, period, formData{}, status, approvedBy, createdAt	本人	週次報告データ（既存移行）
dailyKpi	userId, date, hourlyData[], totals{}, createdAt	本人	日次 KPI データ。時間帯別 6 項目×11 時間帯を含む
kpiTargets	userId, period, kgi{}, conversionRates{}, status, approvedBy	本人→承認	KPI 目標設定。本人提案→上司承認フロー
educationRecords	userId, trainerId, date, theme, content, progress, nextAction	manager	教育記録。trainerId で教育した上司を記録
improveTasks	userId, assignedBy, title, status, deadline, result	manager 以上	改善タスク。status で進捗管理
aiAnalysis	userId, type, input, result, model, createdAt	システム	AI 分析結果キャッシュ
notifications	userId, type, message, read, relatedId, createdAt	システム	通知・アラートデータ
alerts	userId, alertType, threshold, actualValue, severity, createdAt	システム	アラート発火履歴

7. 機能一覧

No.	機能名	概要	フェーズ	対象ロール
F-01	ログイン・認証	メール or Google 認証。ロール・役職に応じた画面制御	Phase 1	全員
F-02	週次報告入力	5 ステップ形式のフォーム。一時保存・閲覧レイアウト対応	Phase 1	全員
F-03	マイヒストリー	過去の報告履歴一覧。編集・PPT/CSV 出力	Phase 1	全員
F-04	KPI 日次入力	時間帯別 6 項目×11 時間帯（8:00	Phase 2	全員

		～19:00)の入力フォーム。合計表示なし(K-01で確認)。修正時は過去データを表示		
F-05	KGI 月次設定	KGI (件数・粗利) と転換率を入力すると必要コール件数等をリアルタイム自動計算。本人提案→上司承認	Phase 2	全員
F-06	KPI ダッシュボード	タブ構成 (KPI 表記・進捗 / 分析・問題点 / 日時指定)。ロール別にデータ範囲を自動切替	Phase 2	全員
F-07	日報管理	日次活動報告。KPI 入力データを活用し追加入力を最小化。入力項目は管理設定から変更可能 (追加・無効化のみ、削除不可)	Phase 2	全員
F-08	教育進捗管理	教育記録・進捗管理。部下別・テーマ別分析	Phase 2	manager 以上
F-09	管理者ダッシュボード	全メンバーの状況一覧・未提出アラート・チーム状況	Phase 1	manager 以上
F-10	PowerPoint 出力	テンプレートに合わせた自動出力	Phase 1	全員
F-11	アラート通知	5 パターンのアラート。個人基準値との乖離を検知	Phase 2	全員
F-12	改善タスク管理	改善タスクの CRUD・期限管理・効果分析	Phase 2	全員
F-08	教育進捗管理	教育記録・進捗管理。部下別・テーマ別分析	Phase 2	manager 以上
F-09	管理者ダッシュボード	全メンバーの状況一覧・未提出アラート・チーム状況	Phase 1	manager 以上
F-10	PowerPoint 出力	テンプレートに合わせた自動出力	Phase 1	全員
F-11	アラート通知	5 パターンのアラート。個人基準値との乖離を検知	Phase 2	全員
F-12	改善タスク管理	改善タスクの CRUD・期限管理・効果分析	Phase 2	全員
F-13	AI KPI 分析	蓄積データから KPI 予測・ボトルネック特定・改善提案	Phase 3	全員
F-14	AI 行動分析	時間帯別データからパフォーマンスパターン分析	Phase 3	全員
F-15	AI 教育フィードバック	個人の課題を分析し具体的改善アクションを自動生成	Phase 3	manager 以上
F-16	Teams 連携	稼働報告の自動取込・通知送信	Phase 3	全員
F-17	外部データ取込	他システムからの CSV/API 自動インポート	Phase 3	executive
F-18	個人成長レポート	入社時からの全実績を時系列で可視化	Phase 3	全員
F-19	組織管理	チーム構成・階層設定・月次変更管理・稼働カレンダー統一設定	Phase 1	executive
F-20	ユーザー管理	アカウント作成・編集・ロール割当	Phase 1	executive
F-21	月次締め入力	翌々月に確定した粗利・件数を入力。本当の着地が翌々月判明のため executive 限定	Phase 2	executive

8. 開発フェーズ計画

Phase 1 : 土台構築（目安：2 週間）

既存の週次報告アプリを拡張し、複数人で利用できる基盤を構築する。

- **Firebase Auth** 導入（3 ロール・10 役職対応）
- **Firestore** セキュリティルール（ロールベース・階層対応）
- 組織管理画面（チーム階層・月次変更管理）
- ユーザー管理画面（アカウント作成・ロール割当）
- 管理者ダッシュボード（ロール別表示切替）
- 週次報告の閲覧レイアウト（詳細ページ）
- **PPT/CSV** 出力の **Firestore** 対応確認
- スマホ対応（レスポンシブデザイン）

Phase 2 : 機能拡張（目安：2～4 週間）

KPI 管理・日報・教育進捗・改善管理・アラートを実装する。

- **KPI** 日次入力フォーム（時間帯別 6 項目×11 時間帯・合計表示なし・修正時過去データ表示）
- **KGI** 月次設定画面（件数・粗利入力・転換率からコール件数等をリアルタイム自動計算・上司承認フロー）
- **KPI** ダッシュボード（タブ構成：KPI 表記進捗 / 分析問題点 / 日時指定・ロール別切替）
- **KPI** 履歴・推移（月・週・期間の選択フィルター）
- 時間帯別 **KPI** 詳細（期間・月の選択フィルター・最大 3 ヶ月表示）
- 日報管理（**KPI** 入力データ活用で追加入力最小化・管理設定と連動した項目変更）
- 月次締め入力（翌々月確定値の粗利・件数入力・**executive** 限定）
- 教育進捗管理（既存 **TrainingTracker** を拡張）
- 改善タスク管理（**CRUD**・期限管理）
- アラート通知（5 パターン・個人基準値対応）
- 通知一覧画面
- プロフィール設定画面

Phase 3 : AI 分析・外部連携（目安：1～2 ヶ月）

蓄積されたデータを活用し、AI 分析と外部システム連携を実装する。

- **Cloud Functions** 構築（サーバーサイド **API** 基盤）
- **AI KPI** 分析（達成予測・ボトルネック分析）
- **AI** 行動分析（時間帯別パフォーマンスパターン分析）
- **AI** 教育フィードバック（個別育成提案）
- **Teams** 連携（**Microsoft Graph API**）
- 外部データ取込（**CSV/API** インポート）
- 個人成長レポート（入社時からの全実績可視化）

9. 既存実装状況

公開 URL: <https://rushup-weekly-report.web.app>
GitHub: <https://github.com/takahashi2412/sales-weekly-report>

実装済み機能	状態	備考
週次報告入力フォーム	完了	5 ステップ形式。一時保存対応
マイヒストリー	完了	履歴一覧・編集・上書き保存・削除
PowerPoint 出力	完了	テンプレート準拠の自動出力
CSV 出力	完了	報告データの CSV ダウンロード
Firestore 移行	完了	localStorage → Firestore に移行済み
Firebase Hosting	完了	SSL デプロイ済み
GitHub 連携	完了	Public リポジトリ。API キーは.env に移行済み
Firestore セキュリティルール	完了	ログイン必須ルール適用済み
ログイン画面	UI のみ	Firebase Auth 連携は未実装
ダッシュボード	UI のみ	モックデータで表示。ロール別切替は未実装
教育進捗管理	一部実装	TrainingTracker.jsx が存在。詳細未確認
チーム管理	一部実装	TeamManagement.jsx が存在。階層対応は未実装
レスポンス対応	一部実装	ハンバーガーメニュー実装済み。一部画面で崩れあり
パンくずリスト	実装済み	汎用 Breadcrumb コンポーネント作成済み

10. 非機能要件

項目	要件
対応デバイス	PC（Chrome, Edge）、スマートフォン（iOS Safari, Android Chrome）
同時利用者数	30～50 名
レスポンス目標	画面遷移 1 秒以内、AI 分析 10 秒以内
データバックアップ	Firestore の自動バックアップ（日次）
セキュリティ	Firebase Auth 認証必須、ロールベースアクセス制御、HTTPS 通信、API キーの.env 管理
可用性	Firebase Hosting / Firestore の標準 SLA（99.95%）に準拠
データ保持期間	無期限（入社時からの全データを蓄積）
バージョン管理	GitHub（Public リポジトリ）で管理

11. 開発体制

役割	担当
設計・相談役	Claude（このチャット / プロジェクト）
実装担当	Google Antigravity（Gemini 3.1 Pro High）
プロジェクト管理	K.t.（株式会社 Rush up）

